

Système de Recommandation sur le Dataset ModCloth

Comparaison du Filtrage Collaboratif et du Filtrage Basé sur le Contenu

Nermine Bacha

Khaldoun Taktak

Télécom SudParis (TSP)

Novembre 2025

Table des matières

1	Introduction	4
1.1	Contexte du Projet	4
1.2	Objectifs	4
1.3	Dataset ModCloth	4
2	Problématique	4
3	Méthodologie	5
3.1	Pipeline de Travail	5
3.2	Prétraitement des Données	5
3.2.1	Chargement	5
3.2.2	Nettoyage	5
3.2.3	Filtrage de la Sparsité	5
3.3	Split Train/Test	6
3.4	Filtrage Collaboratif (CF)	6
3.4.1	Approche Choisie	6
3.4.2	Algorithme de Prédiction	6
3.5	Filtrage Basé sur le Contenu	6
3.5.1	Construction des Profils d'Items	6
3.5.2	Vectorisation TF-IDF	6
3.5.3	Profils Utilisateurs	7
3.6	Métriques d'Évaluation	7
3.6.1	Métriques de Prédiction (CF uniquement)	7
3.6.2	Métriques de Recommandation	7
4	Résultats Expérimentaux	8
4.1	Statistiques du Dataset Après Prétraitement	8
4.2	Distribution des Données	8
4.3	Performances du Filtrage Collaboratif	8
4.4	Performances du Filtrage par Contenu	8
4.5	Comparaison des Modèles	9
5	Analyse et Interprétation	9
5.1	Supériorité du Filtrage par Contenu	9
5.1.1	Sparsité Extrême	9
5.1.2	Distribution des Interactions	9
5.1.3	Nature des Données	9
5.1.4	Qualité des Features Structurées	9
5.2	Limitations du Filtrage Collaboratif	10
5.2.1	Algorithme k-NN Basique	10
5.2.2	Temps de Calcul	10
5.2.3	Problème du Cold Start	10
5.3	Performances Absolues Modestes	10

6	Problèmes Rencontrés et Solutions	10
6.1	Sparsité Excessive	10
6.2	Qualité des Données Textuelles	10
6.3	Temps de Calcul du CF	11
6.4	Déséquilibre des Notes	11
7	Pistes d'Amélioration	11
7.1	Améliorer le Filtrage Collaboratif	11
7.2	Enrichir les Données	11
7.3	Modèle Hybride	12
7.4	Techniques Plus Avancées	12
8	Conclusion	12

1 Introduction

1.1 Contexte du Projet

Les systèmes de recommandation sont devenus omniprésents dans le commerce électronique moderne. Dans le secteur de la mode en ligne, recommander les bons vêtements aux clients est crucial pour améliorer l'expérience utilisateur et augmenter les ventes. Ce projet s'inscrit dans le cadre d'une étude comparative de deux approches fondamentales de recommandation appliquées au domaine de la mode en ligne.

1.2 Objectifs

L'objectif principal de ce projet est de construire et de comparer deux systèmes de recommandation sur le dataset ModCloth Clothing Fit :

1. **Filtrage Collaboratif (CF)** : basé sur les similarités entre utilisateurs ou items à partir des notes données
2. **Filtrage Basé sur le Contenu** : basé sur les caractéristiques textuelles et catégorielles des items

Ce travail vise à déterminer quelle approche est la plus adaptée pour recommander des vêtements dans un contexte de données réelles avec forte sparsité.

1.3 Dataset ModCloth

Le dataset ModCloth Clothing Fit provient de l'UC San Diego McAuley Lab. Il contient des avis d'utilisateurs sur des vêtements avec :

- Des notes d'évaluation (quality)
- Des informations sur l'ajustement (fit : small, fit, large)
- Des catégories de vêtements
- Des avis textuels (review_text, review_summary)
- Des métadonnées morphologiques (taille, poids, etc.)

2 Problématique

Le défi principal de ce projet est de recommander des vêtements dans un contexte difficile. Le dataset ModCloth présente une sparsité extrême : la plupart des utilisateurs n'ont noté que quelques articles sur des milliers disponibles. C'est comme essayer de deviner les goûts de quelqu'un qui n'a commenté que 5 films sur Netflix.

Cette sparsité pose plusieurs problèmes concrets :

- Difficile de trouver des utilisateurs ayant des goûts similaires (trop peu d'overlaps)
- Impossible de recommander aux nouveaux utilisateurs (pas d'historique)
- Les algorithmes classiques de filtrage collaboratif peinent à trouver des patterns

Notre question est simple : face à ces données réelles et imparfaites, quelle méthode marche le mieux ? Le filtrage collaboratif qui se base sur les similarités entre utilisateurs, ou le filtrage par contenu qui utilise les descriptions des vêtements (catégorie, ajustement, avis textuels) ?

3 Méthodologie

3.1 Pipeline de Travail

Notre approche suit le pipeline standard du Machine Learning pour les systèmes de recommandation :

1. **Découverte des données** : exploration et compréhension du dataset
2. **Nettoyage** : gestion des valeurs manquantes et sélection des colonnes
3. **Filtrage de la sparsité** : élimination des utilisateurs/items rares
4. **Construction du dataset** : création des indices et split train/test
5. **Modélisation** : implémentation de CF et filtrage par contenu
6. **Évaluation** : calcul de métriques multiples
7. **Comparaison** : analyse des performances relatives

3.2 Prétraitement des Données

3.2.1 Chargement

Le dataset est téléchargé directement depuis l'URL officielle :

```
1 DATA_URL = "https://mcauleylab.ucsd.edu/public_datasets/data/  
  modcloth/modcloth_final_data.json.gz"  
2 df_raw = pd.read_json(DATA_URL, lines=True, compression='gzip')
```

3.2.2 Nettoyage

Trois stratégies de nettoyage ont été appliquées :

- **Colonnes essentielles** (user_id, item_id, quality) : suppression des lignes avec valeurs manquantes
- **Colonnes textuelles** (review_text, review_summary) : remplacement par chaînes vides
- **Colonnes catégorielles** (category, fit) : remplacement par "unknown"

3.2.3 Filtrage de la Sparsité

Pour réduire la sparsité, nous avons appliqué un filtrage itératif conservant uniquement les utilisateurs et items avec au moins 5 interactions :

```
1 MIN_INTERACTIONS = 5  
2 # Filtrage itératif jusqu'à convergence  
3 while True:  
4     user_counts = df['user_id'].value_counts()  
5     item_counts = df['item_id'].value_counts()  
6     valid_users = user_counts[user_counts >= MIN_INTERACTIONS].  
       index  
7     valid_items = item_counts[item_counts >= MIN_INTERACTIONS].  
       index  
8     df = df[df['user_id'].isin(valid_users)]  
9     df = df[df['item_id'].isin(valid_items)]
```

```

10     # Verifier convergence
11     if len(df) == prev_size: break

```

3.3 Split Train/Test

Un split stratifié par utilisateur (80/20) a été effectué pour simuler un scénario réaliste de prédiction temporelle :

```

1 for user_idx in range(len(unique_users)):
2     user_interactions = ratings_df[ratings_df['user_idx'] ==
3     user_idx].index
4     np.random.shuffle(user_interactions)
5     split_point = int(len(user_interactions) * 0.8)
6     train_indices.extend(user_interactions[:split_point])
7     test_indices.extend(user_interactions[split_point:])

```

3.4 Filtrage Collaboratif (CF)

3.4.1 Approche Choisie

Nous avons implémenté un **CF item-based avec k-NN** et similarité cosinus :

- Construction d'une matrice sparse utilisateur-item
- Utilisation de k=20 plus proches voisins
- Prédiction par moyenne pondérée des similarités

3.4.2 Algorithme de Prédiction

Pour prédire la note qu'un utilisateur u donnerait à un item i :

$$\hat{r}_{u,i} = \frac{\sum_{j \in N(i)} sim(i,j) \cdot r_{u,j}}{\sum_{j \in N(i)} sim(i,j)} \quad (1)$$

où $N(i)$ sont les k items les plus similaires à i que l'utilisateur u a notés, et $sim(i,j)$ est la similarité cosinus entre items i et j .

3.5 Filtrage Basé sur le Contenu

3.5.1 Construction des Profils d'Items

Pour chaque item, nous avons concaténé :

- Les avis textuels (review_text, review_summary)
- La catégorie (répétée 3 fois pour amplifier son importance)
- Le fit (répété 2 fois)

3.5.2 Vectorisation TF-IDF

Transformation des textes en vecteurs TF-IDF :

```

1 tfidf = TfidfVectorizer(
2     max_features=1000,
3     stop_words='english',

```

```

4     ngram_range=(1, 2),
5     min_df=2,
6     max_df=0.8
7 )
8 item_content_matrix = tfidf.fit_transform(item_text_list)

```

3.5.3 Profils Utilisateurs

Le profil d'un utilisateur u est calculé comme la moyenne pondérée des vecteurs TF-IDF des items qu'il a bien notés :

$$\vec{p}_u = \frac{\sum_{i \in I_u} r_{u,i} \cdot \vec{v}_i}{\sum_{i \in I_u} r_{u,i}} \quad (2)$$

où I_u sont les items notés ≥ 4 par l'utilisateur u , $r_{u,i}$ est la note, et $\vec{v}_i$ est le vecteur TF-IDF de l'item i .

3.6 Métriques d'Évaluation

3.6.1 Métriques de Prédiction (CF uniquement)

— **RMSE** (Root Mean Squared Error) :

$$RMSE = \sqrt{\frac{1}{|T|} \sum_{(u,i) \in T} (r_{u,i} - \hat{r}_{u,i})^2} \quad (3)$$

— **MAE** (Mean Absolute Error) :

$$MAE = \frac{1}{|T|} \sum_{(u,i) \in T} |r_{u,i} - \hat{r}_{u,i}| \quad (4)$$

3.6.2 Métriques de Recommandation

— **Precision@k** : proportion d'items pertinents dans les k premiers recommandés

$$Precision@k = \frac{|Recommandés \cap Pertinents|}{k} \quad (5)$$

— **Recall@k** : proportion d'items pertinents capturés parmi tous les pertinents

$$Recall@k = \frac{|Recommandés \cap Pertinents|}{|Pertinents|} \quad (6)$$

Items pertinents : ceux avec note ≥ 4 dans le test set.

4 Résultats Expérimentaux

4.1 Statistiques du Dataset Après Prétraitement

Métrique	Initial	Final
Nombre d'interactions	82,790	16,046
Nombre d'utilisateurs	47,958	2,271
Nombre d'items	1,378	302
Densité de la matrice	0.125%	1.74%
Pourcentage conservé	100%	19.38%

TABLE 1 – Évolution des statistiques après prétraitement

Malgré le filtrage agressif, la sparsité reste très élevée (98.26%), ce qui représente un défi majeur pour le filtrage collaboratif.

4.2 Distribution des Données

- **Interactions par utilisateur** : moyenne = 7.07, médiane = 6.0, min = 5, max = 26
- **Interactions par item** : moyenne = 53.13, médiane = 30.5, min = 5, max = 449

La distribution montre une forte asymétrie avec de nombreux utilisateurs ayant exactement 5 interactions (le minimum filtré) et quelques items très populaires.

4.3 Performances du Filtrage Collaboratif

Métrique	Valeur
RMSE	1.0394
MAE	0.7153
Precision@10	0.0054 (0.54%)
Recall@10	0.0315 (3.15%)
Temps d'évaluation	~18 minutes

TABLE 2 – Résultats du filtrage collaboratif item-based k-NN

4.4 Performances du Filtrage par Contenu

Métrique	Valeur
Precision@10	0.0365 (3.65%)
Recall@10	0.2680 (26.80%)
Temps d'évaluation	~1.5 secondes

TABLE 3 – Résultats du filtrage basé sur le contenu TF-IDF

4.5 Comparaison des Modèles

Métrique	CF	Contenu	Amélioration
Precision@10	0.54%	3.65%	+575%
Recall@10	3.15%	26.80%	+750%
Temps d'exécution	18 min	1.5 s	720× plus rapide

TABLE 4 – Comparaison des performances relatives

Le filtrage par contenu surpasse significativement le filtrage collaboratif sur toutes les métriques de recommandation.

5 Analyse et Interprétation

5.1 Supériorité du Filtrage par Contenu

Plusieurs facteurs expliquent la domination du filtrage par contenu sur ce dataset :

5.1.1 Sparsité Extrême

Avec une densité de seulement 1.74% après filtrage (98.26% de sparsité), le filtrage collaboratif manque cruellement de données pour identifier des patterns robustes. Les overlaps d'interactions entre utilisateurs sont rares, rendant difficile le calcul de similarités fiables.

5.1.2 Distribution des Interactions

La moyenne de 7.07 interactions par utilisateur est très faible. Beaucoup d'utilisateurs ont exactement 5 interactions (le minimum filtré), ce qui ne permet pas au CF de construire des profils utilisateur riches.

5.1.3 Nature des Données

Pour les vêtements, des attributs comme la catégorie (robe, pantalon, etc.) et le fit (ajustement) sont extrêmement informatifs. Un utilisateur qui aime des robes bien ajustées appréciera probablement d'autres robes similaires, une logique bien capturée par le filtrage par contenu.

5.1.4 Qualité des Features Structurées

Malgré que beaucoup d'avis textuels soient vides, les colonnes category et fit sont systématiquement renseignées. La stratégie de répétition de ces features (category $\times$ 3, fit $\times$ 2) a amplifié leur signal dans les vecteurs TF-IDF, rendant le modèle de contenu efficace.

5.2 Limitations du Filtrage Collaboratif

5.2.1 Algorithme k-NN Basique

Le k-NN item-based est une approche simple mais limitée pour des données très sparses. Des méthodes de matrix factorization (SVD, ALS, NMF) auraient probablement mieux géré la sparsité en capturant des patterns latents.

5.2.2 Temps de Calcul

L'évaluation du CF a pris environ 18 minutes contre 1.5 secondes pour le contenu. Cette différence de scalabilité est critique pour un système en production.

5.2.3 Problème du Cold Start

Le CF ne peut pas recommander efficacement pour les nouveaux utilisateurs ou items, alors que le filtrage par contenu peut le faire dès qu'on a une description de l'item.

5.3 Performances Absolues Modestes

Même le meilleur modèle (contenu) n'atteint qu'une Precision@10 de 3.65%. Cela signifie que sur 10 recommandations, moins de 1 est réellement pertinente. Cette performance modeste s'explique par :

- La sparsité résiduelle élevée (98.26%)
- La qualité variable des avis textuels
- L'hétérogénéité des préférences utilisateurs
- Le manque de features riches (images, prix, matériaux, etc.)

Pour un système de production, ces performances seraient insuffisantes. Un modèle hybride combinant CF et contenu serait nécessaire.

6 Problèmes Rencontrés et Solutions

6.1 Sparsité Excessive

Problème : Dataset initial avec 99.87% de sparsité, rendant la modélisation très difficile.

Solution : Filtrage itératif avec seuil de 5 interactions minimum. Cela a amélioré la densité à 1.74% mais au prix de perdre 80% des données.

Limite : Le compromis qualité/quantité reste délicat. Un seuil plus bas (3 ou 4) aurait conservé plus de données.

6.2 Qualité des Données Textuelles

Problème : Environ 8% des avis textuels sont vides après nettoyage initial, beaucoup d'autres sont très courts.

Solution :

- Remplacement des NaN par chaînes vides
- Amplification des features structurées (category $\times$ 3, fit $\times$ 2)
- Utilisation de bigrammes dans TF-IDF (ngram_range=(1,2))

Résultat : Les catégories et le fit deviennent les features dominantes, compensant le manque de texte riche.

6.3 Temps de Calcul du CF

Problème : Évaluation du CF très lente (18 minutes pour 4,033 prédictions).

Solutions appliquées :

- Utilisation de matrices sparse (`scipy.sparse.csr_matrix`)
- Algorithme brute force pour k-NN (optimal pour sparse)
- Limitation de l'évaluation à 100 utilisateurs pour Precision/Recall

Solutions non implémentées :

- Pré-calcul et cache des similarités item-item
- Approximation via LSH (Locality-Sensitive Hashing)
- Parallélisation avec `joblib` ou `multiprocessing`

6.4 Déséquilibre des Notes

Problème : Distribution fortement biaisée vers les notes élevées (4 et 5), notes de 1-2 très rares.

Solution partielle : Seuil de 4.0 pour définir les items "positifs" dans les métriques de ranking.

Non résolu : Techniques de rééquilibrage (sous-échantillonnage) ou métriques pondérées auraient pu être explorées.

7 Pistes d'Amélioration

Nos résultats montrent que le filtrage par contenu surpasse le collaboratif, mais les performances restent modestes (3.65% de précision). Voici comment on pourrait améliorer le système :

7.1 Améliorer le Filtrage Collaboratif

Le k-NN qu'on a utilisé est basique. Des techniques plus avancées comme la factorisation matricielle (SVD) permettent de mieux gérer la sparsité en découvrant des patterns cachés dans les données. Au lieu de comparer directement les utilisateurs, SVD trouve des "facteurs latents" (ex : style casual, formel, sportif) qui expliquent mieux les préférences.

On pourrait aussi essayer de garder plus de données en réduisant le seuil de filtrage à 3 interactions au lieu de 5.

7.2 Enrichir les Données

Le problème principal vient du manque d'information. On pourrait :

- Récupérer les vraies descriptions produits (pas juste les avis utilisateurs)
- Ajouter des images des vêtements et utiliser un réseau de neurones pour extraire des features visuelles
- Intégrer le prix, la marque, les matériaux

7.3 Modèle Hybride

Plutôt que de choisir entre CF et contenu, on pourrait combiner les deux. Par exemple, utiliser le contenu pour les nouveaux utilisateurs (cold start) et progressivement intégrer le CF quand on a assez d'interactions. La formule simple serait :

$$score_{final} = 0.3 \times score_{CF} + 0.7 \times score_{contenu} \quad (7)$$

7.4 Techniques Plus Avancées

Pour aller plus loin, on pourrait utiliser :

- **BERT** pour mieux comprendre les avis textuels (au lieu de simple TF-IDF)
- **Deep Learning** avec des réseaux de neurones qui apprennent automatiquement les meilleures features
- **Modèles séquentiels** qui prennent en compte l'ordre des achats (ex : après un pantalon, recommander une chemise assortie)

8 Conclusion

Ce projet nous a permis de comparer deux approches classiques des systèmes de recommandation sur des données réelles de e-commerce de mode. Le résultat principal est clair : sur le dataset ModCloth avec sa sparsité extrême (98.26%), le filtrage par contenu surpasse largement le filtrage collaboratif (Recall de 26.80% vs 3.15%). Cette différence s'explique par le manque d'interactions communes entre utilisateurs, qui empêche le CF de trouver des similarités robustes, alors que le filtrage par contenu peut exploiter les attributs structurés des vêtements (catégorie, ajustement) qui sont bien renseignés. Cependant, même le meilleur modèle n'atteint qu'une précision de 3.65%, ce qui reste faible et montre les limites de ces approches basiques face à la complexité des préférences humaines en matière de mode.

Pour améliorer significativement les performances, un modèle hybride combinant les deux approches avec des techniques plus avancées (factorisation matricielle, embeddings BERT, deep learning) serait nécessaire, ainsi qu'un enrichissement des données avec des images et des descriptions produits complètes.